



Consultas Avanzadas y Relaciones

Clase 4 - Consultas agrupadas (Parte I)



¿Qué aprenderás en esta sesión?

En esta sesión aprenderemos a agrupar datos en SQL utilizando funciones de agregación y la cláusula GROUP BY, herramientas fundamentales para el análisis estadístico de grandes volúmenes de información estructurada.

Exploraremos funciones como MIN, MAX, AVG y COUNT, aplicándolas en contextos reales como la educación, el comercio y la administración pública.

Analizaremos cómo estas funciones nos permiten sintetizar información relevante para la toma de decisiones, qué errores debemos evitar al implementarlas y cómo su correcto uso mejora la eficiencia y claridad en la presentación de resultados.

Desafío inicial



Trabajas como analista en una entidad gubernamental que monitorea programas de capacitación para jóvenes. Se te ha solicitado generar un reporte que muestre:

- El promedio de calificaciones por cada programa.
- Cuántos estudiantes participaron por cada modalidad (presencial/online).
- La nota máxima y mínima registrada en cada curso.

Para ello, necesitas conocer y dominar funciones de agregación y agrupamiento. ¿Qué pasos debes seguir para obtener estos datos a partir de una tabla que contiene cientos de registros? ¿Cómo interpretarías los resultados si algunos grupos tienen menos participantes?

Funciones de agregación en SQL: MIN, MAX, AVG, COUNT

¿Cómo responderías rápidamente cuántos estudiantes se inscribieron en un curso, cuál fue la calificación más alta o cuál es el promedio general de notas usando una base de datos SQL?

Las funciones de agregación permiten aplicar cálculos sobre un conjunto de filas y devolver un único resultado por grupo. Son especialmente útiles para obtener estadísticas resumidas sin necesidad de exportar los datos a otro sistema.



MIN()

Encuentra el valor mínimo en un conjunto de datos



MAX()

Encuentra el valor máximo en un conjunto de datos



AVG()

Calcula el promedio de valores numéricos



COUNT()

Cuenta el número de filas o valores no nulos

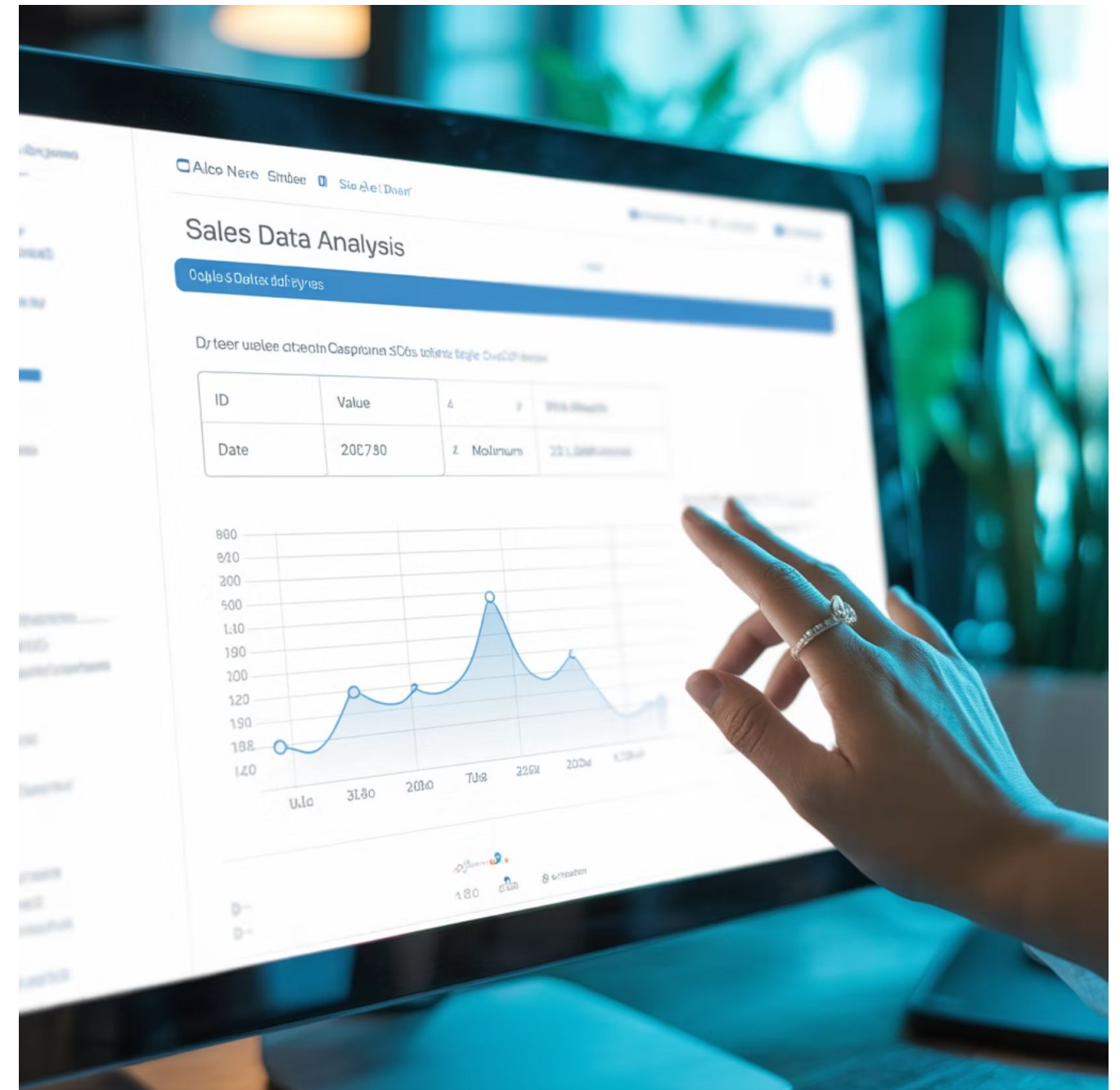
Función MIN() y MAX()

- MIN() devuelve el valor mínimo de una columna numérica, de fecha o incluso texto (orden lexicográfico).
- MAX() devuelve el valor máximo.

Ejemplo aplicado:

```
SELECT  
    MIN(calificacion) AS nota_minima,  
    MAX(calificacion) AS nota_maxima  
FROM evaluaciones;
```

Contexto productivo: útiles para monitorear extremos de rendimiento, fechas más antiguas o recientes, o identificar productos más caros/baratos en comercio.



Errores comunes y buenas prácticas

Errores comunes:

- Aplicarlas sin filtrar previamente por condiciones relevantes (ej. programa o modalidad).
- Suponer que los valores extremos son representativos sin validarlos con otros indicadores.

Buenas prácticas:

- Usar alias (AS) para nombrar las columnas resultantes.
- Complementarlas con filtros WHERE cuando se desea limitar el análisis.
- Validar outliers que podrían distorsionar los valores extremos.

Función AVG()

- Calcula el promedio aritmético de una columna numérica.

Ejemplo aplicado:

```
SELECT
    AVG(calificacion) AS promedio_general
FROM evaluaciones
WHERE modalidad = 'Online';
```

Contexto productivo: muy usada en análisis de desempeño académico, satisfacción de clientes, productividad, entre otros.



Errores comunes y buenas prácticas con AVG()

Errores comunes:

- No considerar que excluye los valores NULL.
- Malinterpretar el promedio como representativo sin considerar la cantidad de datos.
- Comparar promedios entre grupos muy dispares en tamaño.

Buenas prácticas:

- Complementar AVG() con COUNT() para interpretar el contexto.
- Usar filtros para excluir valores atípicos que distorsionen el promedio.
- Considerar desviación estándar si se desea un análisis más robusto.

Métrica clave sugerida:

Número de registros usados en el promedio (COUNT()).

Función COUNT()

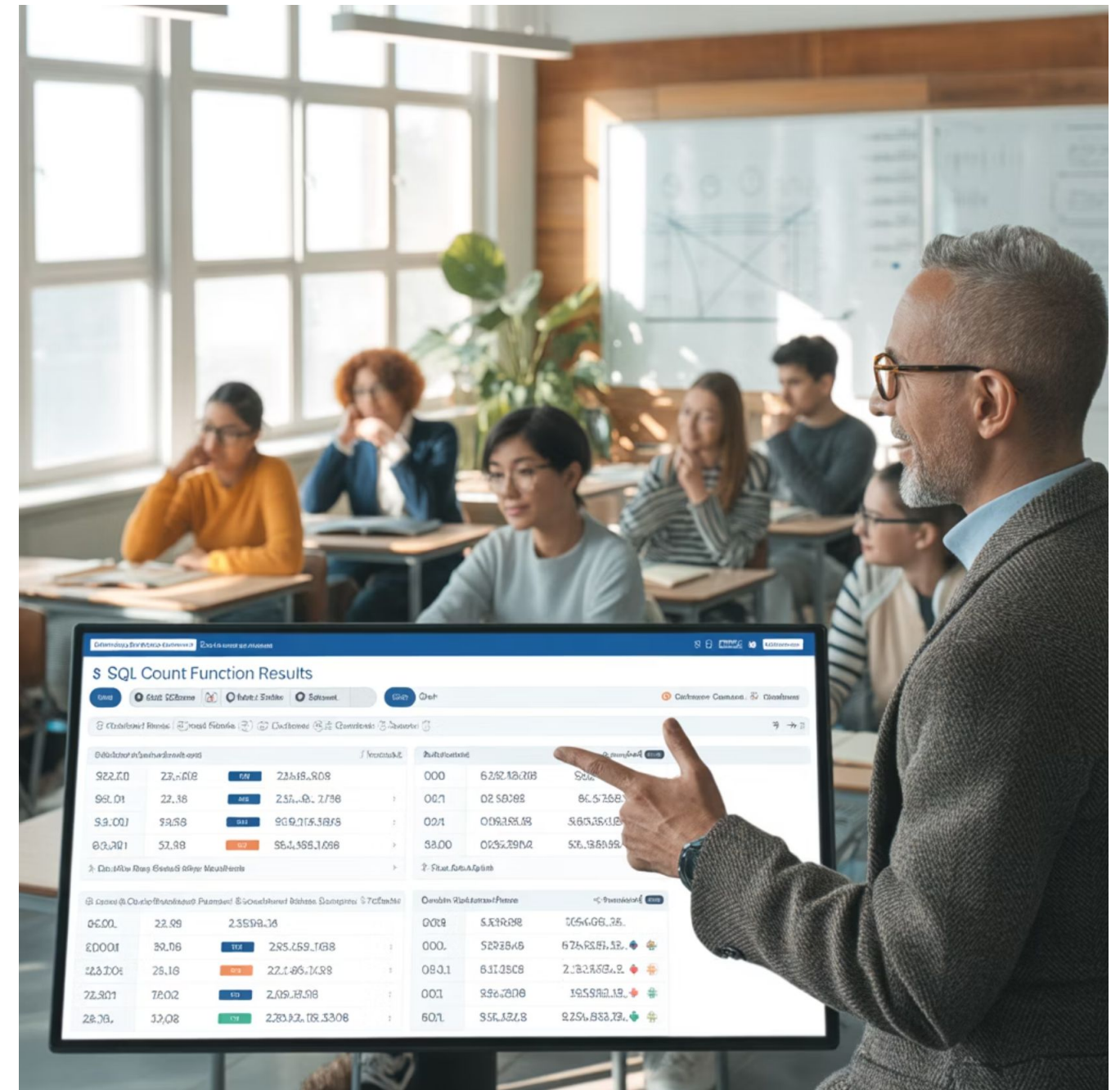
- Devuelve el número de filas o valores no nulos.

Ejemplo aplicado:

```
SELECT  
    COUNT(*) AS total_estudiantes  
FROM estudiantes;
```

Variantes:

- COUNT(*): cuenta todas las filas.
- COUNT(columna): cuenta solo las filas donde columna no es NULL.



Errores comunes y buenas prácticas con COUNT()

Errores comunes:

- Suponer que COUNT(*) y COUNT(columna) entregan el mismo valor.
- Aplicar COUNT() a columnas no representativas del análisis deseado.

Buenas prácticas:

- Usar COUNT() junto con GROUP BY para análisis por segmento.
- Nombrar claramente el indicador mediante alias.
- Validar si la columna contiene valores nulos antes de aplicar COUNT(columna).

Métrica relacionada:

Proporción de nulos respecto al total:

```
SELECT
    COUNT(*) AS total,
    COUNT(columna) AS no_nulos,
    COUNT(*) - COUNT(columna) AS
nulos
FROM tabla;
```

Las funciones de agregación nos ofrecen una primera aproximación para sintetizar datos, sin embargo, muchas veces necesitamos aplicar estas funciones por grupo, por ejemplo, por modalidad, por curso o por región. Para ello, usamos GROUP BY, una herramienta esencial para el análisis comparativo y segmentado. Veamos cómo funciona en detalle.

Actividad guiada:

**Generando reportes de
desempeño por programa de
capacitación**



Objetivo de la actividad

Aplicar funciones de agregación y cláusulas de agrupamiento (GROUP BY, HAVING) para generar reportes segmentados a partir de datos de estudiantes, comprendiendo el uso correcto de funciones como AVG, COUNT, MIN, MAX y evitando errores comunes.



Situación problema

Formas parte del equipo de análisis de datos de un servicio público de empleo que gestiona programas de formación técnica.

Te solicitan preparar un resumen con los siguientes indicadores:



Promedio de calificaciones por programa de formación.



Total de estudiantes por modalidad (online/presencial).



Nota mínima y máxima por curso.



Listado de programas que tuvieron más de 10 estudiantes.

Necesitas organizar y presentar esta información usando funciones de agregación y la cláusula GROUP BY, identificando posibles errores si no aplicas correctamente las condiciones de agrupación.

Instrucciones

1. Crear la tabla base (modo demostrativo):

```
CREATE TABLE evaluaciones (  
  estudiante VARCHAR(50),  
  programa VARCHAR(50),  
  modalidad VARCHAR(20),  
  calificacion DECIMAL(3,1)  
);  
  
INSERT INTO evaluaciones VALUES  
  ('Ana', 'Programación', 'Online', 6.5),  
  ('Luis', 'Programación', 'Online', 5.8),  
  ('María', 'Redes', 'Presencial', 6.1),  
  ('Carlos', 'Redes', 'Online', 5.9),  
  ('Elena', 'Administración', 'Presencial', 6.7),  
  ('Jorge', 'Administración', 'Presencial', 6.9),  
  ('Lucía', 'Programación', 'Presencial', 6.3),  
  ('Daniel', 'Redes', 'Online', 5.4),  
  ('Sofía', 'Programación', 'Online', 5.7),  
  ('Pedro', 'Administración', 'Online', 6.1);
```



Consultas a ejecutar

1

Promedio de calificaciones por programa:

```
SELECT
    programa,
    AVG(calificacion) AS promedio
FROM evaluaciones
GROUP BY programa;
```

2

Total de estudiantes por modalidad:

```
SELECT
    modalidad,
    COUNT(*) AS cantidad
FROM evaluaciones
GROUP BY modalidad;
```

3

Nota mínima y máxima por programa:

```
SELECT
    programa,
    MIN(calificacion) AS nota_min,
    MAX(calificacion) AS nota_max
FROM evaluaciones
GROUP BY programa;
```

4

Programas con más de 2 estudiantes:

```
SELECT
    programa,
    COUNT(*) AS total
FROM evaluaciones
GROUP BY programa
HAVING COUNT(*) > 2;
```

Reflexionar con el grupo: ¿Qué patrones observan en los resultados?, ¿Qué decisiones podrían tomarse a partir de estos datos?, ¿Qué pasaría si se agregara una nueva modalidad? ¿Se actualizarían correctamente los datos?

Materiales necesarios



- Entorno de práctica (SQLite, DBeaver, PostgreSQL, u otro compatible con SQL estándar).
- Código base entregado por el relator (creación de tabla y registros).

Actividad práctica autónoma:

**Análisis agrupado de
rendimiento académico**



Objetivo de la actividad

Aplicar funciones de agregación y cláusulas de agrupamiento (GROUP BY, HAVING) sobre un conjunto de datos simulado del ámbito educativo, con el fin de generar reportes segmentados por programa y modalidad, y reflexionar sobre el valor de estas técnicas en entornos reales.

Contexto

Formas parte del equipo de monitoreo de una institución que ofrece cursos de formación técnica.

Cuentas con una base de datos que registra estudiantes, programas cursados, modalidad de asistencia y calificaciones obtenidas.

Tu tarea es generar reportes estadísticos por segmento para ayudar al área pedagógica a identificar fortalezas y posibles focos de mejora.



Instrucciones para desarrollar la actividad de forma autónoma

1. Crea la tabla y carga los datos:

```
CREATE TABLE evaluaciones (  
    estudiante VARCHAR(50),  
    programa VARCHAR(50),  
    modalidad VARCHAR(20),  
    calificacion DECIMAL(3,1)  
);  
  
INSERT INTO evaluaciones VALUES  
    ('Pedro', 'Programación', 'Online', 6.1),  
    ('Laura', 'Administración', 'Presencial', 6.5),  
    ('Ignacio', 'Redes', 'Online', 5.8),  
    ('Fernanda', 'Programación', 'Presencial', 6.7),  
    ('Javiera', 'Administración', 'Presencial', 6.0),  
    ('Tomás', 'Redes', 'Online', 5.9),  
    ('Valeria', 'Programación', 'Online', 6.3),  
    ('Carlos', 'Administración', 'Online', 6.2),  
    ('Sofía', 'Redes', 'Presencial', 6.6),  
    ('Luis', 'Programación', 'Online', 5.5);
```

2. Consulta el promedio de calificaciones por programa.

```
SELECT  
    programa,  
    AVG(calificacion) AS promedio  
FROM evaluaciones  
GROUP BY programa;
```



Consultas a ejecutar

1

Total de estudiantes por modalidad

```
SELECT
    modalidad,
    COUNT(*) AS
total_estudiantes
FROM evaluaciones
GROUP BY modalidad;
```

2

Calificación mínima y máxima por programa

```
SELECT
    programa,
    MIN(calificacion) AS
nota_min,
    MAX(calificacion) AS
nota_max
FROM evaluaciones
GROUP BY programa;
```

3

Programas con más de 3 estudiantes

```
SELECT
    programa,
    COUNT(*) AS total
FROM evaluaciones
GROUP BY programa
HAVING COUNT(*) > 3;
```



Reflexiona y responde

- ¿Qué programa muestra mayor dispersión entre la nota mínima y máxima?
- ¿Qué modalidad concentra más estudiantes?
- ¿Por qué es útil aplicar HAVING en vez de WHERE al contar por grupo?
- ¿Cómo podrías mejorar el reporte para presentar estos datos a tu equipo?

Resumen

Aprendimos a utilizar funciones de agregación (MIN, MAX, AVG, COUNT) para obtener estadísticas descriptivas sobre columnas numéricas y categóricas.

Comprendimos la utilidad de GROUP BY para agrupar registros y aplicar funciones por categoría o segmento.

Identificamos diferencias clave entre WHERE y HAVING en el contexto de agrupaciones.

Aplicamos agrupamientos múltiples y analizamos buenas prácticas para interpretar resultados de forma clara y precisa.



Próxima sesión...

En la siguiente sesión profundizaremos en el filtrado de resultados agrupados utilizando la cláusula HAVING.

A diferencia de WHERE, que actúa antes del agrupamiento, HAVING permite aplicar condiciones directamente sobre los resultados agregados.

Esta herramienta es clave para generar reportes que excluyan grupos poco representativos o que no cumplan ciertos criterios analíticos.

Preguntas de cierre

¿Cuál es la diferencia entre MIN() y MAX() y en qué situaciones conviene utilizar cada una?

¿Qué precauciones debes tener al interpretar un AVG() en un grupo con pocos elementos?

¿Por qué es importante aplicar GROUP BY correctamente antes de usar funciones agregadas en el SELECT?

¿Cuál es la diferencia entre aplicar una condición en WHERE y en HAVING?

¿Qué utilidad tiene realizar agrupamientos múltiples con más de una columna?

Referencias bibliográficas

- McKinney, W. (2022). Python for Data Analysis: Data Wrangling with Pandas, NumPy, and Jupyter (3rd ed.). O'Reilly Media.
- PostgreSQL Global Development Group. (2024). PostgreSQL 16 Documentation. <https://www.postgresql.org/docs/>
- Coronel, C., & Morris, S. (2017). Database Systems: Design, Implementation, & Management (13th ed.). Cengage Learning.
- Gobierno de Chile. (s.f.). ChileValora: Marco de Cualificaciones Técnico Profesional. <https://www.chilevalora.cl/marco-de-cualificaciones/>